

Practical 7

PHP Form Processing with Server-Side Validation and CSV Storage

Objective

1. Create an HTML registration form.
2. Submit data using POST.
3. Receive form data using PHP.
4. Validate data on the **server side**.
5. Sanitize user input.
6. Store data in a CSV file.
7. Display success/error messages.
8. Display stored records.

Project Structure

Create a folder named: practical7

Inside the XAMPP htdocs folder: C:\xampp\htdocs\practical7

Create below mentioned files:

practical7/

```
|
|— index.php
|— process.php
|— records.php
|— registrations.csv
```

Note: Initially, registrations.csv can be empty.

Step 1 — Create the Registration Form

Create **index.php**.

```
<!DOCTYPE html>
<html>
<head>
  <title>Registration Form</title>

  <style>
    body {
      font-family: Arial;
      background-color: #f2f2f2;
    }

    .container {
      width: 400px;
      margin: 50px auto;
      background: white;
      padding: 25px;
      border-radius: 10px;
    }
  </style>
</head>
<body>
```

```
input, select {
    width: 100%;
    padding: 10px;
    margin: 8px 0 15px;
    box-sizing: border-box;
}

input[type="submit"] {
    background-color: #333;
    color: white;
    border: none;
    cursor: pointer;
}
</style>
</head>

<body>

<div class="container">

    <h2>Student Registration</h2>

    <form action="process.php" method="POST">

        <label>Name:</label>
        <input type="text" name="name">

        <label>Email:</label>
        <input type="text" name="email">

        <label>Mobile:</label>
        <input type="text" name="mobile">

        <label>Course:</label>
        <select name="course">
            <option value="">Select Course</option>
            <option value="B.Tech IT">B.Tech IT</option>
            <option value="B.Tech CSE">B.Tech CSE</option>
            <option value="BCA">BCA</option>
            <option value="MCA">MCA</option>
        </select>

        <input type="submit" value="Register">

    </form>

</div>

</body>
</html>
```

The most important part is: <form action="process.php" method="POST">

Means

When the user clicks Register, send the form data to process.php

Method used

method="POST"

means

Send the form data using the HTTP POST method. (Ans of 1st Question)

Understanding: name

For example: <input type="text" name="name">

The important part is: name="name"

PHP will use this name to retrieve the value.

For example: \$_POST["name"] Similarly name="email"

So

HTML name attribute → PHP \$_POST key.

Step 2 — Create process.php

```
<?php

// 1. Check whether form is submitted using POST
if ($_SERVER["REQUEST_METHOD"] == "POST") {

    // 2. Get form data
    $name = $_POST["name"] ?? "";
    $email = $_POST["email"] ?? "";
    $mobile = $_POST["mobile"] ?? "";
    $course = $_POST["course"] ?? "";

    // 3. Remove unnecessary spaces
    $name = trim($name);
    $email = trim($email);
    $mobile = trim($mobile);
    $course = trim($course);

    // 4. Sanitize input
    $name = htmlspecialchars($name);
    $email = htmlspecialchars($email);
    $mobile = htmlspecialchars($mobile);
    $course = htmlspecialchars($course);

    // 5. Validation
    $errors = [];

    if ($name == "") {
        $errors[] = "Name is required.";
    }

    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors[] = "Enter a valid email address.";
    }

    if (!preg_match("/^[0-9]{10}$/", $mobile)) {
        $errors[] = "Mobile number must contain exactly 10 digits.";
    }

    if ($course == "") {
        $errors[] = "Please select a course.";
    }

    // 6. Check for errors
    if (count($errors) > 0) {

        echo "<h2>Registration Failed</h2>";

        echo "<ul>";

        foreach ($errors as $error) {
            echo "<li>" . $error . "</li>";
        }

        echo "</ul>";

        echo '<a href="index.php">Go Back</a>';

    } else {

        // 7. CSV file name
```

```

$file = "registrations.csv";

// 8. Open CSV file
$handle = fopen($file, "a");

// 9. Check whether file opened successfully
if ($handle === false) {

    echo "Error: Unable to open CSV file.";

} else {

    // 10. Add header if file is empty
    if (filesize($file) == 0) {

        fputcsv($handle, [
            "Name",
            "Email",
            "Mobile",
            "Course"
        ]);
    }

    // 11. Store student data
    fputcsv($handle, [
        $name,
        $email,
        $mobile,
        $course
    ]);

    // 12. Close file
    fclose($handle);

    // 13. Success message
    echo "<h2>Registration Successful!</h2>";

    echo "<p>Your data has been saved successfully.</p>";

    echo '<a href="index.php">Register Another Student</a><br>';

    echo '<a href="records.php">View All Records</a>';

}
} else {

    echo "Invalid Request.";
}

?>

```

Check POST

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {
```

Meaning: Check whether the request came using the POST method.

Receiving Form Data

```
$name = $_POST["name"] ?? "";
```

`$_POST` is a PHP array containing data submitted using POST.

For example, if the user enters:

Name: Rahul

then

`$_POST["name"]`

contains:

Rahul

The ?? "" means:

If the value doesn't exist, use an empty string.

Why trim()?

`$name = trim($name);`

Suppose the user enters:

Rahul Patel

`trim()` removes unnecessary spaces from the beginning and end.

Result:

Rahul Patel

Sanitization

`$name = htmlspecialchars($name);`

This converts special HTML characters into safe text.

For example, a user might enter:

`<script>alert("Hello")</script>`

`htmlspecialchars()` prevents the value from being interpreted as HTML when we display it.

So, Sanitization makes input safer to use/display.

Validation

`$errors = [];`

This will store all validation errors.

For example:

`$errors[] = "Name is required.";`

means:

Add this error message to the `$errors` array.

Name Validation

```
if ($name == "") {  
    $errors[] = "Name is required."  
}
```

If the name is empty:

Name is required.

is added to the error list.

Email Validation

```
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
```

PHP has a built-in function called:

`filter_var()`

We use:

`FILTER_VALIDATE_EMAIL`

to check whether the email has a valid format.

Example:

rahul@gmail.com

Valid.

But:

rahul@ is invalid

Mobile Number Validation

We use:

```
if (!preg_match("/^[0-9]{10}$/", $mobile)) {
```

means

`^` → beginning

`[0-9]` → numbers from 0 to 9

`{10}` → exactly 10 numbers

`$` → end

Therefore:

9876543210 is valid. But 98765 is invalid

Course Validation

```
if ($course == "") {  
    $errors[] = "Please select a course.";  
}
```

If the student doesn't select a course, an error is generated.

Checking Errors

```
if (count($errors) > 0) {  
    count() tells us how many errors exist.
```

For example:

```
$errors = [  
    "Name is required.",  
    "Enter a valid email address."  
];
```

Then

```
count($errors)
```

returns

2

So the condition becomes: $2 > 0$ which is true

Displaying Errors

```
foreach ($errors as $error) {  
    echo "<li>" . $error . "</li>";  
}
```

foreach goes through each error one by one.

For example:

Name is required.

Enter a valid email address.

Mobile number must contain exactly 10 digits.

Step 3 — Display CSV Records

Create records.php

```
<?php

$file = "registrations.csv";

echo "<h2>Registered Students</h2>";

if (!file_exists($file)) {

    echo "<p>No records found.</p>";

} else {

    $handle = fopen($file, "r");

    echo "<table border='1' cellpadding='10'>";

    echo "<tr>";
    echo "<th>Name</th>";
    echo "<th>Email</th>";
    echo "<th>Mobile</th>";
    echo "<th>Course</th>";
    echo "</tr>";

    // Skip header row
    fgetcsv($handle);

    while (($data = fgetcsv($handle)) !== false) {

        echo "<tr>";

        echo "<td>" . htmlspecialchars($data[0]) . "</td>";
        echo "<td>" . htmlspecialchars($data[1]) . "</td>";
        echo "<td>" . htmlspecialchars($data[2]) . "</td>";
        echo "<td>" . htmlspecialchars($data[3]) . "</td>";

        echo "</tr>";
    }

    echo "</table>";

    fclose($handle);
}

echo '<br><a href="index.php">Back to Registration</a>';

?>
```

Complete Working Flow

Student

|

↓

Fill Registration Form

|

↓

Click Register

|

↓

index.php submits

POST

|

↓

process.php

|

↓

Receive \$_POST data

|

↓

trim() / sanitize

|

↓

Server-side Validation

|

┌───────────┴───────────┐

↓

Invalid

↓

Display Errors

↓

Valid

↓

Open CSV File

|

↓

Write Record

|



Close File

|



Success Message

|



records.php

|



Display Records